Pre-Lab Report: Data Acquisition and Digital Conversion Feedback Control

Written by: Demarce Williams DSW9740

1. Number System Conversions

(a) Hexadecimal to Decimal and Binary ($AC43_{16}$)

Hexadecimal to Decimal:

16^3	16^2	16^1	16^0
4096	256	16	1
A	C	4	3

$(4096 \times 10) + (256 \times 12) + (16 \times 4) + (3 \times 1) = 40960 + 3072 + 64 + 3 = 44099_{10}$

$\therefore AC43_{16} = 44099_{10}$

Hexadecimal to Binary: Converting each hex digit to a 4-bit binary number:

A = 10

2^3	2^2	2^1	2^0
8	4	2	1
1	0	1	0

C = 12

2^3	2^2	2^1	2^0
8	4	2	1
1	1	0	0

4

2^3	2^2	2^1	2^0
8	4	2	1
0	1	0	0

3

2^3	2^2	2^1	2^0
8	4	2	1
0	0	1	1

$\therefore AC43_{16} = 1010\ 1100\ 0100\ 0011_2$

(b) Decimal to Binary and Hexadecimal (9999d)

Decimal to Binary:

The largest power of 2 that fits into 9999_{10} is $8192_{10} = 2^{13}$
$9999_{10} - 8192_{10} = 1807_{10}$

The largest power of 2 that fits into 1807_{10} is $1024_{10} = 2^{10}$
$1807_{10} - 1024_{10} = 783_{10}$

The largest power of 2 that fits into 783_{10} is $512_{10} = 2^{9}$
$783_{10} - 512_{10} = 271_{10}$

The largest power of 2 that fits into 271_{10} is $256_{10} = 2^{8}$
$271_{10} - 256_{10} = 15_{10}$

The largest power of 2 that fits into 15_{10} is $8_{10} = 2^{3}$
$15_{10} - 8_{10} = 7_{10}$

The largest power of 2 that fits into 7_{10} is $4_{10} = 2^{2}$
$7_{10} - 4_{10} = 3_{10}$

The largest power of 2 that fits into 3_{10} is $2_{10} = 2^{1}$
$3_{10} - 2_{10} = 1_{10}$
The largest power of 2 that fits into 1_{10} is $1_{10} = 2^{0}$

.: $9999_{10} = 10\ 0111\ 0000\ 1111_{2}$

Decimal to Hexadecimal: Using repeated division by 16:

16	**9999**	**R**
16	624	15
16	39	0
16	2	7
	0	2

.: $9999_{10} = 270F_{16}$

2. ADC/DAC Quantization Analysis

(a) 12-bit ADC Analysis (4.67 V Input)

Parameters: n=12 bits, Range 0−5 V, vin=4.67 V

Note: *I did two approaches here. In the manual you gave formulas where $N = 2^n$, but in the example you used $N = 2^n-1$ for ADC conversion. I think this suggest that there are different standards to perform these conversions.*

Using Formulas Given:

ADC : $x = \left\lfloor 2^n \frac{v - v_{min}}{v_{max} - v_{min}} \right\rfloor$ DAC: $v = x \frac{v_{max} - v_{min}}{2^n} + v_{min}$

Given: n = 12 bits , v = 4.67 V, v_{min} = 0 V, v_{max} = 5V

The corresponding digital value:

$$x = \left\lfloor 2^{12} \frac{4.67 - 0}{5 - 0} \right\rfloor = 3825$$

The corresponding analog voltage:

$$v = 3825 \frac{5 - 0}{2^{12}} + 0 = 4.66919\, V$$

Quantization error:

$$Error = 4.67 - 4.66919 = 0.81\, mV$$

$$\%Error = \frac{4.67 - 4.66919}{4.67} \times 100 = 0.0173\ \%$$

Based on Example:

ADC : $x = \left\lfloor (2^n - 1) \frac{v - v_{min}}{v_{max} - v_{min}} \right\rfloor$ DAC: $v = x \frac{v_{max} - v_{min}}{2^n} + v_{min}$

Given: n = 12 bits , v = 4.67 V, v_{min} = 0 V, v_{max} = 5V

The corresponding digital value:

$$x = \left\lfloor (2^{12} - 1) \frac{4.67 - 0}{5 - 0} \right\rfloor = 3824$$

The corresponding analog voltage:

$$v = 3824 \frac{5 - 0}{2^{12}} + 0 = 4.66797V$$

Quantization error:

$$Error = 4.67 - 4.66797 = 2.03\, mV$$

$$\%Error = \frac{4.67 - 4.66797}{4.67} \times 100 = 0.0435\ \%$$

The voltage is not exactly equal to the applied voltage. As observed, using different standards (N−1 for ADC vs N for DAC) introduces a systematic gain error, effectively increasing the quantization error.

(b) 10-bit DAC Output (Target 4.3 V)

Parameters: n=10 bits (N=1024), Range 0−5 V

ADC : $x = \left\lfloor 2^{10} \frac{4.3-0}{5-0} \right\rfloor = 880_{10}$

16	**880**	**R**
16	55	0
16	3	7
	0	3

Hence, hexadecimal output = 370_{16}

3. Implementation of *Read_Voltage*

The function utilizes a polling mechanism to synchronize the software execution with the hardware sampling rate managed by the background *callback*.

```
// --- Function Initialisations ---
float64 Read_Voltage();
```

```
// --- Function Definitions ---

float64 Read_Voltage() {
    // Wait until the background callback sets sample_ready to 1
    while (sample_ready != 1) {
        // Waits for the hardware to finish a sample
    }

    // Acknowledge the sample by resetting the flag to 0
    sample_ready = 0;

    // Return the value stored by the callback
    return read_voltage[0];
}
```

4. Main DAQ Control Program

The following program configures the system as a "Voltage Follower," mirroring analog input to analog output.

```
#include <iostream>
#include "daq.h" // DAQ header file

using namespace std;

// --- Main Program ---

int main() {
    // Define parameters for configuration
    float64 SAMPLE_RATE = 1000.0;   // 1 kHz frequency
    int32 TIMEOUT = 5;              // 5 second timeout
    float64 V_MIN = -10.0;          // Minimum voltage range
    float64 V_MAX = 10.0;           // Maximum voltage range

    // Configure Analog Input (AI 0)
    AI_Configuration(SAMPLE_RATE, TIMEOUT, V_MIN, V_MAX);

    // Configure Analog Output (AO 0)
    AO_Configuration(V_MIN, V_MAX);

    cout << "DAQ System initialized. Starting voltage follower..." << endl;
```

```
    // Control Loop
    while (true) {
        // Read the input voltage once a sample is ready using Read_Voltage()
        float64 V_IN = Read_Voltage();

        // Write the same voltage to the output (AO 0)
        Write_Voltage(V_IN);

    }

    return 0;
}
```

The codes above can also be found in the C++ document submitted with this report.